sotOS

Verified Microkernel

Architecture Document

Philosophy, Design & Implementation

Version 0.1.0

March 2026

Microkernel + Selective Exokernel Bypass

Rust ▪ Capability Security ▪ Zero-Copy IPC ▪ Formal Verification

Contents

1 Philosophy: A Third Way **3**
1.1 Why Verification Matters . . . 3

2 Architecture Overview **3**
2.1 What Lives Where . . . 4

3 The Five Kernel Primitives **4**
3.1 Primitive 1: Thread Scheduling . . . 5
3.2 Primitive 2: IRQ Virtualization . . . 5
3.3 Primitive 3: IPC Core . . . 5
3.3.1 Endpoint Message Transfer . . . 5
3.3.2 Channel Ring Buffer . . . 6
3.4 Primitive 4: Capability Management . . . 6
3.4.1 Capability Derivation Tree (CDT) . . . 6
3.4.2 Capability Types . . . 6
3.5 Primitive 5: Frame Allocation . . . 6

4 Security Model: Defense in Depth **7**
4.1 Confinement . . . 7
4.2 CHERI Integration (Future) . . . 7

5 Memory Model **8**
5.1 Physical Memory: Kernel Domain . . . 8
5.2 Virtual Memory: Userspace Domain . . . 8
5.3 Higher-Half Kernel . . . 8

6 IPC in Detail **8**
6.1 Syscall Interface . . . 8
6.2 Typed Channels (Future) . . . 9

7 Storage: Transactional Object Store **9**

8 Distribution Model **10**
8.1 Location Transparency with Explicit Failure . . . 10
8.2 Failure Domain Hierarchy . . . 10

9 Boot Chain & Trust **10**

10 Current Implementation Status **11**
10.1 Project Structure . . . 11
10.2 What Works Today . . . 11
10.3 Boot Output . . . 12

11 Comparison with Existing Systems **12**

12 Observability **12**

13 Hot-Patching & Live Update **13**

14 LUCAS: Legacy UNIX Compatibility Abstraction System **13**

14.1 Architecture 13
14.2 Components 14
14.3 Execution Flow: `write(fd, buf, size)` 14
14.4 Security: Confinement by Design 14
14.5 Comparison with Other Compatibility Layers 15

15 Summary: Design Principles **15**

1 Philosophy: A Third Way

Operating systems have traditionally followed two opposing philosophies:

- **Microkernels** (seL4, L4, Minix3): The kernel abstracts and mediates. Services ask the kernel for permission to do anything.
- **Exokernels** (MIT Exokernel, Unikernels): The kernel abstracts nothing — it exposes raw hardware and lets applications manage their own resources.

sotOS is neither. It is a **verified microkernel with selective exokernel bypass**.

The Core Idea

The kernel is a formally verified microkernel managing exactly **five primitives**. Virtual memory is handled by a *userspace* server. Applications that need maximum performance (databases, ML inference) can *bypass* the standard memory server and manage their own physical frames directly — as long as the kernel can verify they own those frames through capabilities.
The kernel guarantees isolation, not policy.

This creates a spectrum: most applications use the standard VMM server and get the safety of managed memory. Performance-critical applications opt out and get raw hardware access — but only to resources they hold capabilities for.

1.1 Why Verification Matters

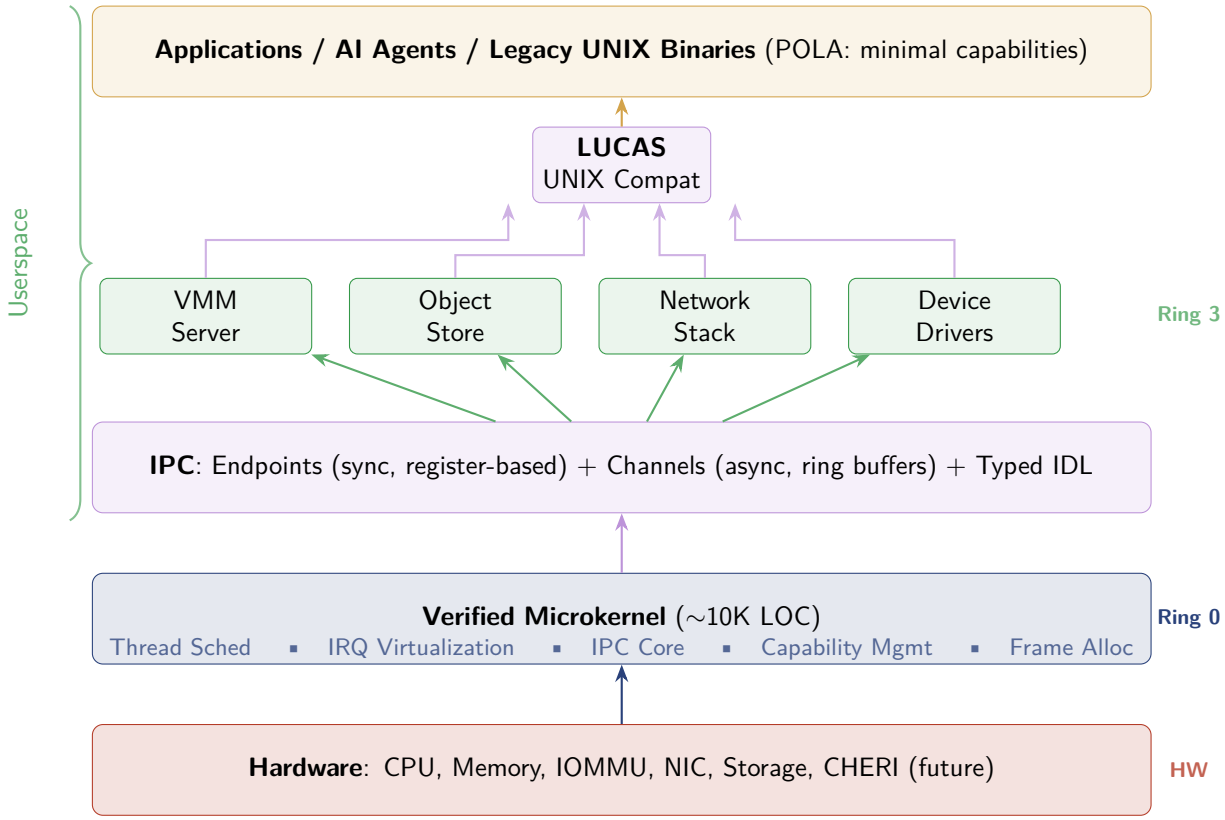
Rust eliminates memory errors, but not logic errors. A scheduler written in Rust can be memory-safe and still deadlock. A capability check can be memory-safe and still have an off-by-one that grants too much access.

For a kernel of ∼10K lines, formal verification is tractable. seL4 proved it: you can mathematically guarantee that the kernel never dereferences a null pointer, never overflows a buffer, and that capability enforcement is correct.

Layer	Tool	Guarantee
Kernel	Rust + Verus/Prusti	Memory safety + functional correctness
Critical services	Rust + TLA+/Spin	No deadlocks, liveness, fairness
Drivers/services	Rust + fuzzing	Memory safety + empirical robustness

The 70% figure (Microsoft/Google) covers memory bugs that Rust eliminates. The remaining 30% are logic bugs — attacked with formal verification in the kernel and contained by capability isolation everywhere else.

2 Architecture Overview

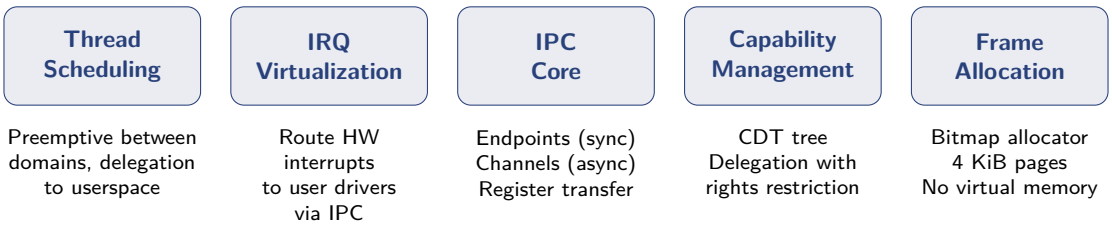


2.1 What Lives Where

Component	Responsibility	Ring
Thread scheduler	Preemptive scheduling between domains	0
IRQ virtualizer	Route hardware interrupts to userspace	0
IPC core	Endpoint rendezvous, channel setup	0
Capability manager	CDT, delegation, revocation	0
Frame allocator	Physical memory allocation	0
VMM server	Page tables, virtual memory policy	3
Object store	Transactional storage, OIDs	3
Network stack	TCP/IP, socket API	3
Device drivers	Hardware I/O, DMA	3
Scheduling domains	Intra-domain thread scheduling	3

3 The Five Kernel Primitives

The kernel implements exactly five primitives. Everything else — filesystems, networking, device drivers, virtual memory policy — runs in userspace.

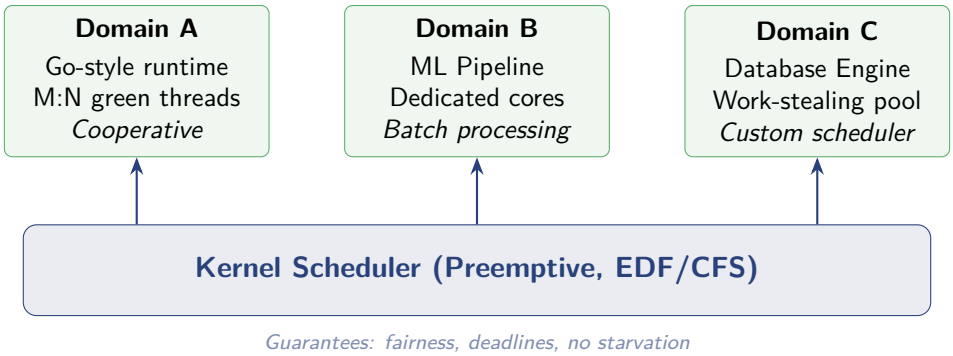


3.1 Primitive 1: Thread Scheduling

The kernel scheduler is **preemptive** and **priority-based**. It schedules *between scheduling domains* — coarse-grained decisions involving at most dozens of entities. Within each domain, userspace implements its own scheduler (cooperative green threads, work-stealing pools, dedicated cores, etc.).

Why Hierarchical Scheduling?

- The kernel makes few, fast decisions (which domain gets the CPU).
- Domain-internal scheduling is specialized per workload.
- If a domain misbehaves (exceeds its quantum), the kernel preempts it.
- Most context switches happen *within* a domain — in userspace, with zero syscalls.



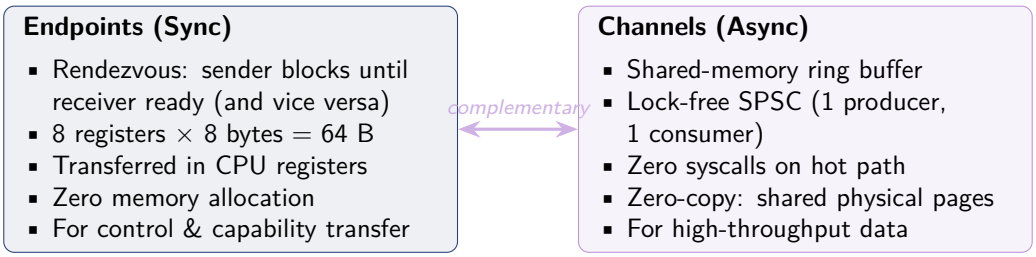
3.2 Primitive 2: IRQ Virtualization

Hardware interrupts are **not handled by the kernel** beyond acknowledging them. Instead, the kernel delivers IRQ notifications to userspace driver threads via IPC endpoints.

1. A driver registers for an IRQ: `sys_irq_register(irq_cap, endpoint_cap)`
2. Hardware fires the IRQ → kernel handler sends notification to the endpoint
3. Driver thread (blocked on `recv`) wakes up and handles the interrupt
4. Driver acknowledges: `sys_irq_ack(irq_cap)` → kernel unmasks the IRQ line

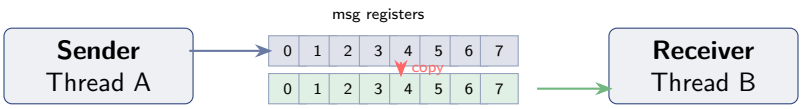
3.3 Primitive 3: IPC Core

Two mechanisms, optimized for different use cases:



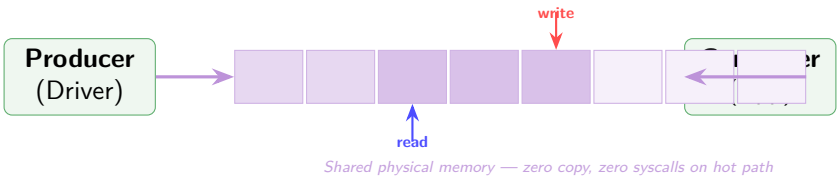
3.3.1 Endpoint Message Transfer

When sender and receiver rendezvous, message registers are copied directly between thread contexts during the context switch — zero intermediate buffers:



3.3.2 Channel Ring Buffer

For bulk data transfer, shared-memory channels avoid syscalls entirely on the hot path:

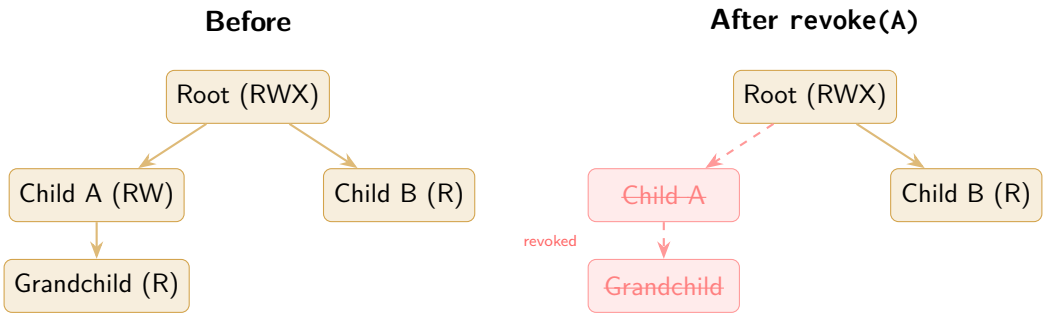


3.4 Primitive 4: Capability Management

Every kernel object is accessed through a **capability** — an opaque handle with associated rights. Processes never see raw pointers to kernel objects; they see tokens that the kernel translates.

3.4.1 Capability Derivation Tree (CDT)

When a capability is *delegated*, the kernel records the parent-child relationship in a tree. Revoking a parent automatically invalidates all descendants:



3.4.2 Capability Types

Type	Object	Rights
`Memory`	Physical frame(s)	Read, Write, Execute, Grant, Revoke
`Endpoint`	IPC endpoint	Send, Receive, Grant
`Thread`	Thread control	Suspend, Resume, Destroy, Grant
`Irq`	Hardware IRQ line	Register, Ack, Grant
`IoPort`	x86 I/O port range	Read, Write, Grant

3.5 Primitive 5: Frame Allocation

The kernel manages physical memory as 4 KiB frames using a bitmap allocator. Each bit represents one frame: 0 = free, 1 = allocated.

Critical Design Decision

The kernel does **not** manage virtual memory (page tables, mappings). That is the job of the userspace VMM server. The kernel only:

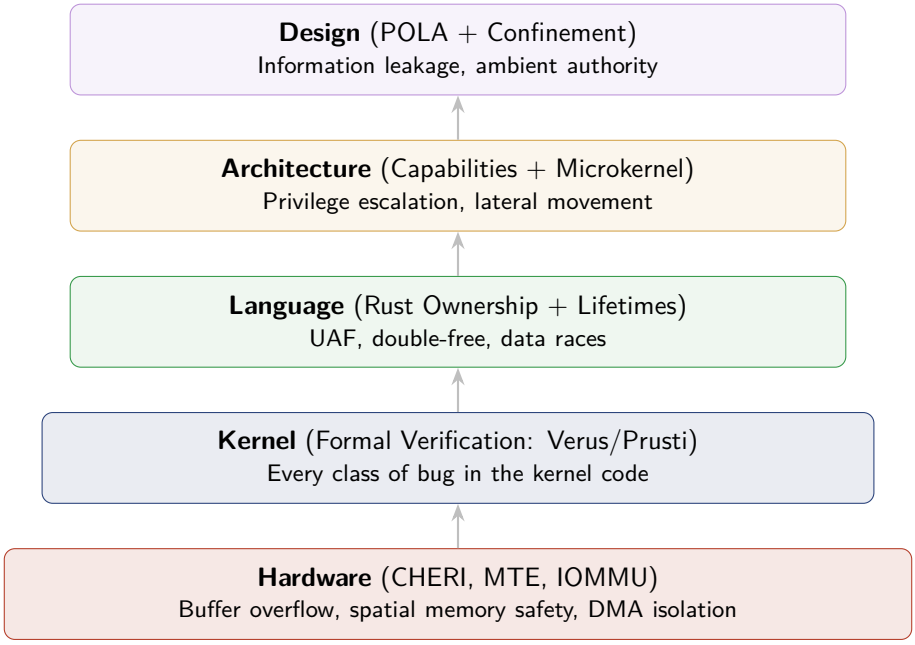
1. Allocates/frees physical frames

2. Verifies that `sys_map()` calls reference frames the caller owns (via capability)
3. Ensures kernel memory is never mapped to userspace

This is what enables the exokernel bypass: a process with frame capabilities can map memory however it wants, without going through the VMM.

4 Security Model: Defense in Depth

sotOS implements four layers of defense. Each layer catches what the layers above it miss:



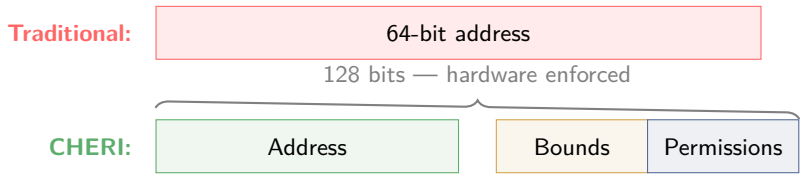
4.1 Confinement

A process can be launched with the guarantee that it **cannot leak information** outside the capabilities it was given. This is essential for sandboxing untrusted code: plugins, extensions, downloaded code.

The property is enforced structurally: if a process holds no `Send` capability to any external endpoint, it physically cannot communicate with the outside world. No ambient authority exists — there are no global namespaces, no file paths accessible without a capability, no implicit shared state.

4.2 CHERI Integration (Future)

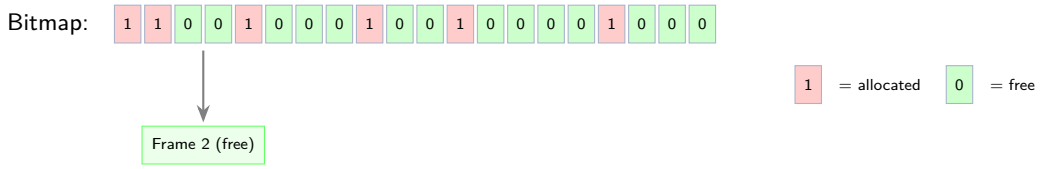
ARM Morello implements CHERI — Capability Hardware Enhanced RISC Instructions. Instead of raw 64-bit pointers, CHERI uses 128-bit "fat pointers" carrying bounds and permission metadata *in hardware*. A buffer overflow doesn't just get caught by software — the CPU traps before the instruction executes.



5 Memory Model

5.1 Physical Memory: Kernel Domain

The kernel sees physical memory as a flat array of 4 KiB frames. At boot, the Limine bootloader provides a memory map, and the kernel builds a bitmap:



5.2 Virtual Memory: Userspace Domain

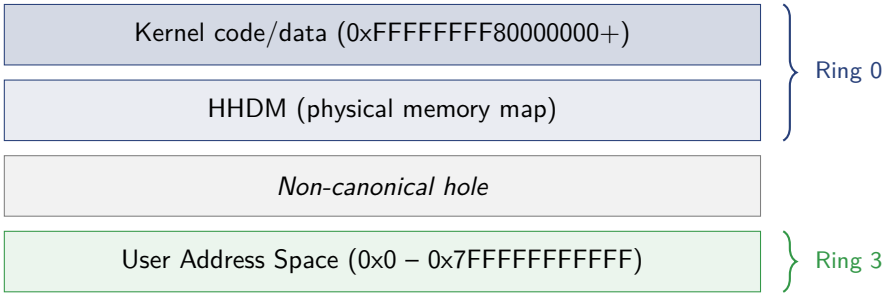
The kernel does **not** maintain page tables for user processes. Instead:

1. The VMM server receives page fault notifications from the kernel
2. It decides how to handle the fault (allocate, map, swap, deny)
3. It calls `sys_map(addr_space_cap, vaddr, frame_cap, flags)` to install mappings
4. The kernel verifies the capability and writes the page table entry

No overcommit: if you ask for 1 GiB, you get 1 GiB. No OOM killer.

5.3 Higher-Half Kernel

The kernel is linked at virtual address `0xFFFFFFFF80000000` (higher half). The Limine bootloader establishes a Higher Half Direct Map (HHDM) that identity-maps all physical memory at a high offset, giving the kernel direct access to any physical address.



6 IPC in Detail

6.1 Syscall Interface

The entire kernel API is exposed through these syscalls:

Number	Syscall	Description
0	`Yield`	Yield current timeslice
1	`Send`	Send message to endpoint
2	`Recv`	Receive message from endpoint
3	`Call`	Send + Recv atomically (RPC)
10	`EndpointCreate`	Create new IPC endpoint
20	`FrameAlloc`	Allocate physical frame
21	`FrameFree`	Free physical frame
22	`Map`	Map frame into address space
23	`Unmap`	Unmap virtual page
30	`CapGrant`	Delegate capability (with restriction)
31	`CapRevoke`	Revoke capability + all derivatives
40	`ThreadCreate`	Create new thread
41	`ThreadDestroy`	Destroy thread
50	`IrqRegister`	Bind IRQ to endpoint
51	`IrqAck`	Acknowledge IRQ

All syscalls require at least one capability as argument. There is **no** way to access any resource without the appropriate capability.

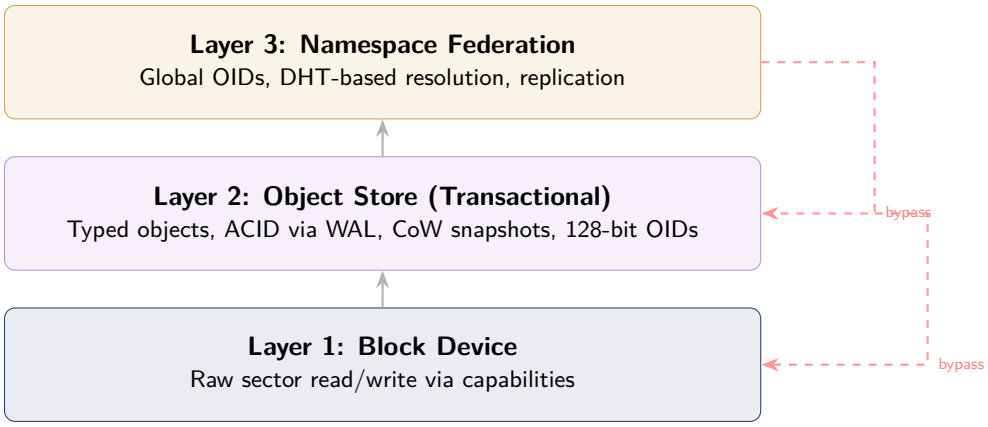
6.2 Typed Channels (Future)

Each channel will have a schema defined in an Interface Definition Language (IDL). The IDL compiler generates Rust stubs that serialize/deserialize automatically with compile-time type checking and zero runtime parsing:

```
// sotos-idl definition (future)
interface FileServer {
    fn open(path: String, flags: u32) → Result<FileHandle, Error>;
    fn read(handle: FileHandle, buf: &mut [u8]) → Result<usize, Error>;
    fn write(handle: FileHandle, data: &[u8]) → Result<usize, Error>;
    fn close(handle: FileHandle) → Result<(), Error>;
}
```

7 Storage: Transactional Object Store

Three layers, each providing an escape hatch to the one below:



Properties of the Object Store:

- **Typed objects**: Every object has a schema. No unstructured blobs at this layer.
- **Transactional**: ACID via Write-Ahead Log. Crash at any point → consistent recovery.
- **Versioned**: Copy-on-Write at the page level. Every mutation creates an implicit snapshot.
- **POSIX compat**: A translation server can expose the store as a traditional filesystem.

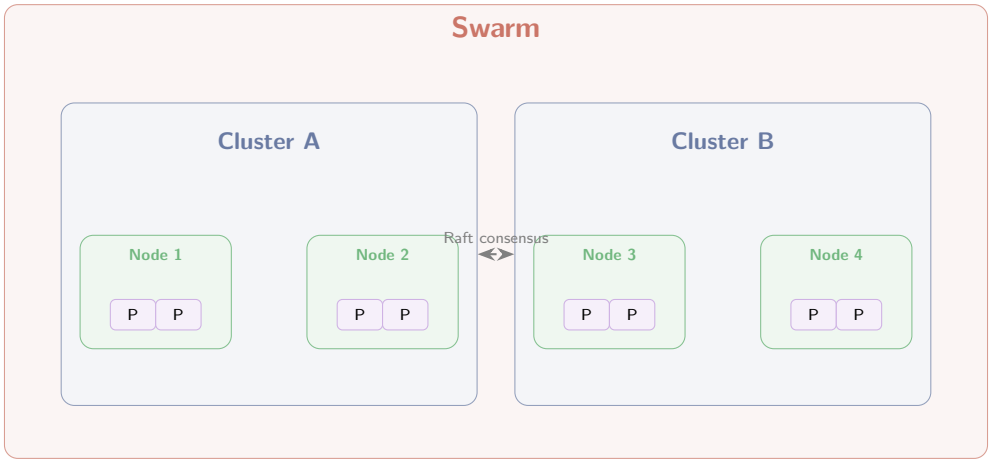
8 Distribution Model

8.1 Location Transparency with Explicit Failure

The same IPC interface works locally and remotely. But remote calls return `Result<T, RemoteError>` — the programmer **knows** the call can fail with timeouts, disconnections, and network partitions.

This is fundamentally different from CORBA/DCOM which pretended the network didn't exist. sotOS acknowledges the Fallacies of Distributed Computing.

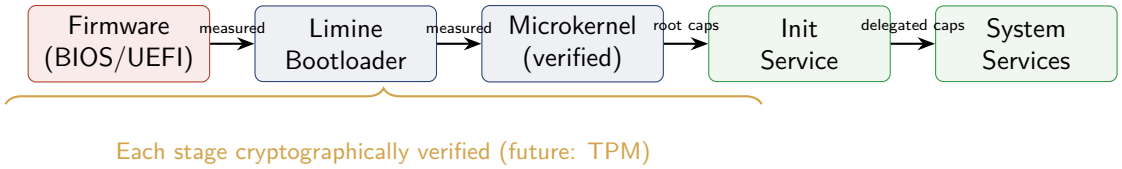
8.2 Failure Domain Hierarchy



Each level has its own failure detection and recovery:

Level	Failure	Recovery
Process	Crash or misbehavior	Restart in milliseconds
Node	Hardware failure	Migrate services to another node
Cluster	Network partition	Operate in partitioned mode (eventual consistency)
Swarm	Cluster disconnect	Autonomous operation, reconcile on reconnect

9 Boot Chain & Trust



The init service is the **only** process that receives root capabilities. All other services receive a restricted subset — the Principle of Least Authority (POLA).

10 Current Implementation Status

10.1 Project Structure

```
sotOS/
  kernel/src/
    main.rs      — entry point, Limine boot
    panic.rs     — panic handler
    arch/x86_64/
      serial.rs  — COM1 UART driver
      gdt.rs     — GDT + TSS
      idt.rs     — IDT + exception handlers
    mm/
      frame.rs   — bitmap frame allocator
    cap/
      table.rs   — capability table + CDT
    ipc/
      endpoint.rs — sync IPC endpoints
    sched/
      thread.rs  — TCB + CpuContext
  libs/sotos-common/src/
    lib.rs       — syscall ABI, error codes
  scripts/
    mkimage.py   — disk image builder
```

10.2 What Works Today

Component	Status	
Limine boot (BIOS)	Kernel loads and executes	✓
Serial output	COM1 UART, `kprintln!` macro	✓
GDT + TSS	64-bit segments, double fault IST	✓
IDT	Breakpoint, double fault, GPF, PF	✓
Frame allocator	Bitmap, alloc/free, HHDM	✓
Capability table	Insert, lookup, CDT revocation	✓
IPC endpoints	Create only (no send/recv yet)	~
Scheduler	TCB defined (no context switch yet)	~
Syscall ABI	Defined in sotos-common	~
QEMU boot pipeline	`just run` end-to-end	✓
Timer interrupt	Not implemented	✗
Context switching	Not implemented	✗
Syscall entry/exit	Not implemented	✗
Ring 3 (userspace)	Not implemented	✗
Shared-memory channels	Not implemented	✗
VMM server	Not implemented	✗

10.3 Boot Output

When the kernel boots in QEMU, the serial output shows:

```
limine: Loading executable `boot():/boot/sotos-kernel`...
sotOS v0.1.0 -- microkernel booting...
[ok] GDT
[ok] IDT
  frames: 268435456 total, 55982 free (218 MiB usable)
[ok] Frame allocator
  capability table ready (max 4096 entries)
[ok] Capabilities
  scheduler: preemptive round-robin (single core)
[ok] Scheduler
Kernel ready. Primitives: sched, irq, ipc, cap, frame_alloc
Halting -- no init service yet.
```

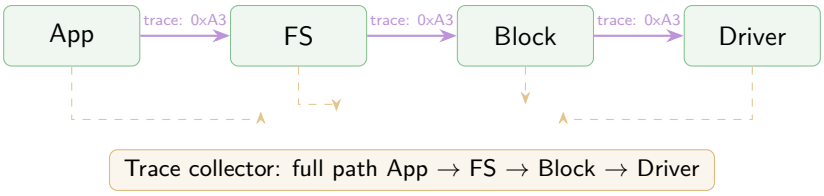
11 Comparison with Existing Systems

Property	Legacy UNIX	seL4	Fuchsia	sotOS
Architecture	Monolithic	Microkernel	Microkernel	Micro + exo bypass
Language	C	C + Isabelle	C++ / Rust	Rust + Verus
Verification	No	Full proof	Partial	Target: full
VM in kernel	Yes	Yes	Yes	**No** (userspace)
Capabilities	No (DAC/MAC)	Yes	Yes (handles)	Yes + CHERI
IPC	Pipes/sockets	Sync endpoints	Channels	Endpoints + channels
Zero-copy	splice/io_uring	No	VMOs	Ring buffers
Scheduling	CFS/RT	Fixed priority	Fair scheduler	Hierarchical + domains
POSIX compat	Native	Manual	Starnix layer	**LUCAS**

12 Observability

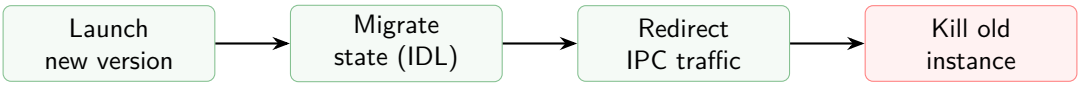
Every service in sotOS exposes metrics and traces natively:

- **Distributed tracing**: Every IPC message carries a trace ID. A request can be traced across N services end-to-end.
- **Metrics**: Counters, histograms, gauges in a standard format. The system aggregates automatically.
- **Audit log**: Every capability invocation can be logged, creating a complete forensic trail.



13 Hot-Patching & Live Update

Userspace services can be updated without system restart:



Zero downtime. The kernel uses kexec for its own (rare) updates.

14 LUCAS: Legacy UNIX Compatibility Abstraction System

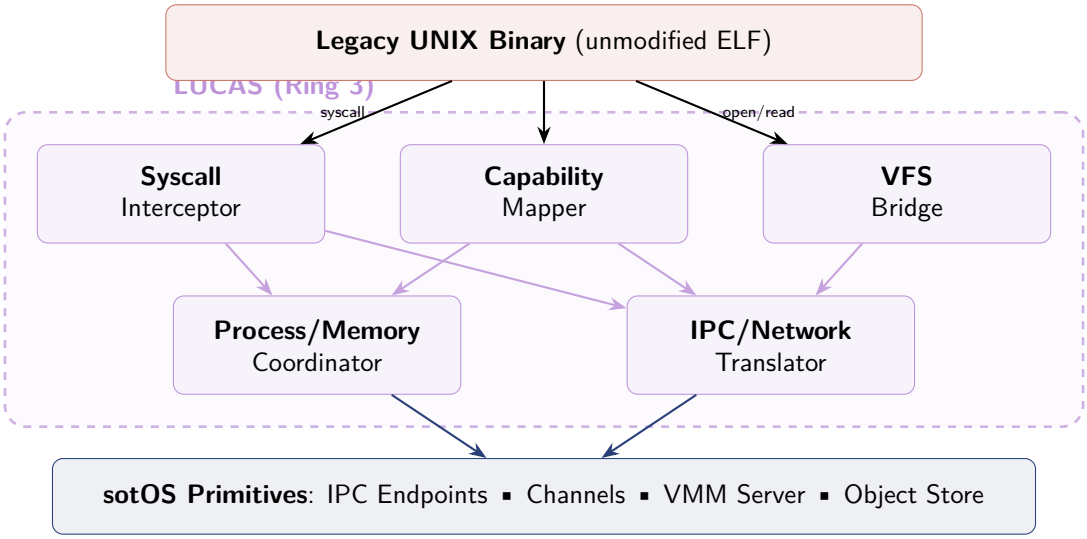
LUCAS is a userspace subsystem that allows **unmodified Linux/POSIX ELF binaries** to run natively on sotOS. Unlike a hypervisor that virtualizes hardware, LUCAS is a *paravirtualizer of interfaces* — it translates monolithic UNIX semantics into sotOS's decentralized, capability-based primitives.

LUCAS in One Sentence

LUCAS presents the illusion of a complete Linux system to legacy binaries, while every operation is actually mediated by capabilities and sotOS IPC underneath. The guest binary thinks it's on Linux; it's actually sandboxed in a capability jail.

14.1 Architecture

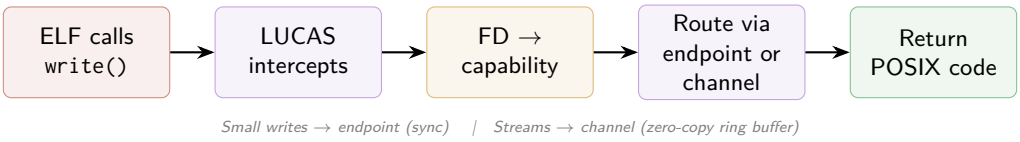
LUCAS is not a monolithic emulator — it is a set of coordinated Ring 3 servers:



14.2 Components

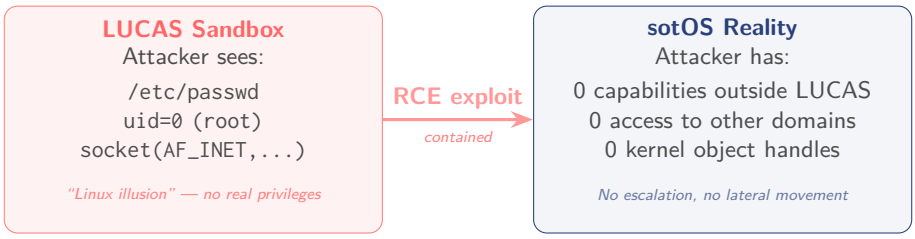
Component	Role
Syscall Interceptor	Captures `syscall`/`int 0x80` from the guest binary in Ring 3. Redirects to LUCAS translation engine instead of the kernel. Maps Linux syscall numbers to LUCAS handlers.
Capability Mapper	Maintains an internal table mapping Linux file descriptors (FDs) and PIDs to real sotOS capabilities. Emulates UID/GID permissions on top of the capability model. Hides the CDT from the guest binary.
VFS Bridge	Intercepts POSIX file operations (`open`, `read`, `write`, `stat`) and translates them to Object Store (Layer 2) transactions. Exposes typed objects as flat byte streams. Synthesizes `/proc` and `/sys`.
Process/Memory Coordinator	Emulates `fork()` (CoW via VMM), `execve()` (ELF parsing + segment mapping), `mmap()`/`brk()` (delegated to VMM server). Creates threads via `sys_thread_create`.
IPC/Network Translator	Translates UNIX pipes → ring buffer channels, signals → endpoint messages, sockets → channels (zero-copy for high throughput). Emulates `epoll`/`select` over sotOS IPC.

14.3 Execution Flow: `write(fd, buf, size)`



14.4 Security: Confinement by Design

From a cybersecurity perspective, LUCAS operates as a **strict isolation sandbox**:



Why LUCAS Is Fundamentally More Secure Than Linux

In sotOS, there is **no ambient authority**. A Linux binary running inside LUCAS can only interact with resources that LUCAS explicitly provides capabilities for. Even if an attacker achieves Remote Code Execution (RCE) by exploiting a vulnerability in an emulated web server, the damage is confined:

- The compromised binary holds no native capabilities to interact with other sotOS protection domains or the underlying hardware.

- The attacker is trapped inside the "Linux illusion" — they can corrupt the emulated environment but cannot escape the capability jail.
- Privilege escalation is structurally impossible: there is no `root`, no `/dev/mem`, no `CAP_SYS_ADMIN` — only capability tokens that the kernel validates on every operation.

14.5 Comparison with Other Compatibility Layers

Property	WSL1	Wine	Fuchsia Starnix	LUCAS
Runs in	Kernel	Userspace	Kernel runner	Userspace
Target ABI	Linux	Win32	Linux	Linux/POSIX
Isolation	Weak	None	Moderate (zircon)	**Full (caps)**
Zero-copy IPC	No	No	VMOs	Ring buffers
Native perf	Good	Variable	Good	Good (bypass)
Security model	Shared kernel	Shared proc	Handles	Capability jail

15 Summary: Design Principles

Minimal kernel
Only 5 primitives in Ring 0

Verify everything
Formal proofs for the kernel

Capabilities only
No ambient authority, ever

Zero-copy IPC
Ring buffers, no intermediate copies

Policy in userspace
VM, scheduling, FS — all Ring 3

Explicit failure
Network errors are not hidden

Escape hatch
Exokernel bypass for who needs it

LUCAS compat
Run legacy UNIX in a capability jail

sotOS — The kernel guarantees isolation, not policy.

v0.1.0 ▪ March 2026